## Experiment No: - 1.4

**Student Name:** Raghav Gupta
**UID:** 23BCS12959
**Branch:** BE-CSE
**Section/Group:** 23BCS_620_B
**Semester:** 7th
**Date of Performance:** 14/08/2026
**Subject Name:** Competitive Coding - 3
**Subject Code:** 23CSP-403

### 1. Aim:

Write a program to find the **length of the longest substring without repeating characters** using the **Sliding Window optimization technique**.

### 2. Objective:

- To understand the concept of the sliding window technique.
- To find the longest substring containing unique characters.
- To avoid unnecessary repeated traversal of the string.
- To analyze the time and space complexity of the optimized approach.

### 3. Algorithm:

1. Take a string `s` as input.
2. Initialize:
   - `left = 0` to represent the starting point of the window.
   - `maxLength = 0` to store the maximum length.
   - An array/map to store the last position of each character.
3. Traverse the string using `right` from left to right.
4. If the current character has already appeared **inside the current window**, move `left` to one position after its previous occurrence.
5. Calculate the current window length:
   ```
   right - left + 1
   ```
6. Update `maxLength` if the current window is larger.
7. Print the longest substring length.

**4. Code Implementation:**

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String s = sc.next();

        Map<Character, Integer> lastIndex = new HashMap<>();

        int left = 0;
        int maxLength = 0;

        for (int right = 0; right < s.length(); right++) {

            char current = s.charAt(right);

            if (lastIndex.containsKey(current) &&

                lastIndex.get(current) >= left) {

                left = lastIndex.get(current) + 1;
            }
            lastIndex.put(current, right);

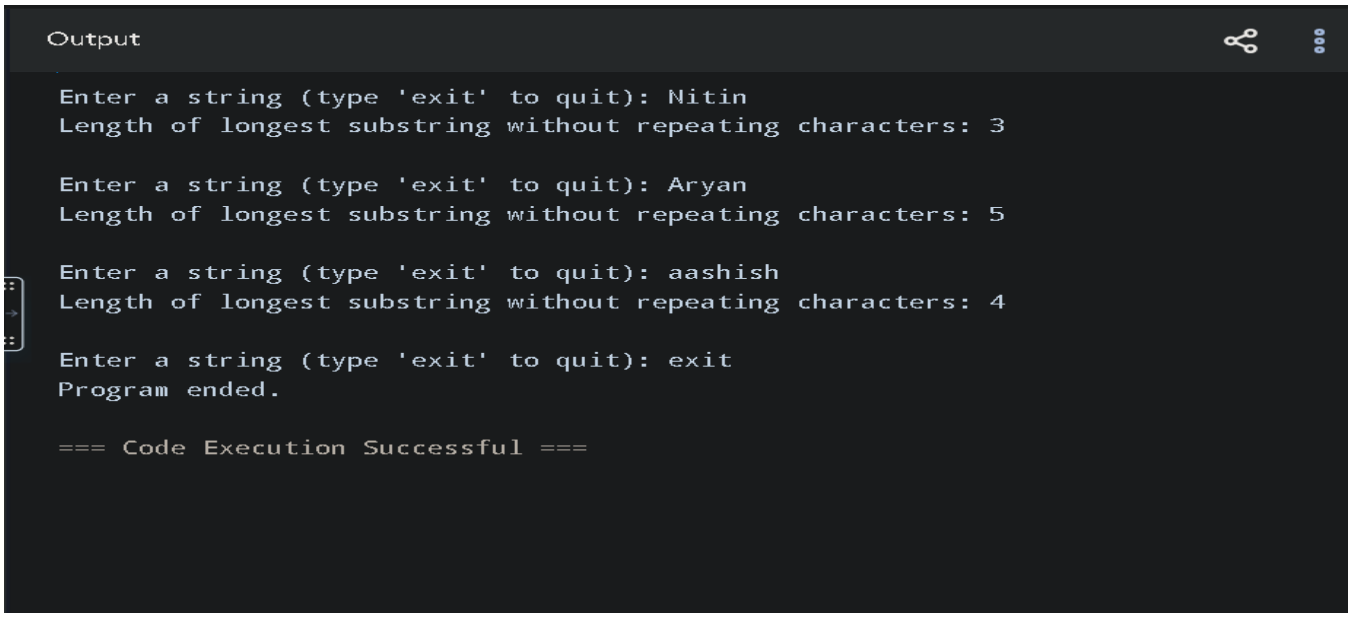
            int currentLength = right - left + 1;

            maxLength = Math.max(maxLength, currentLength);
        }

        System.out.println(
            "Length of longest substring without repeating characters: "
```

```
            + maxLength
        );
        sc.close();
    }
}
```

## 5. Output:




## 6. Learning Outcomes:

1. Understand the **Sliding Window** technique.
2. Identify and solve substring-related problems efficiently.
3. Use `unordered_map` to track character positions.
4. Understand how an $O(n^2)$ brute-force solution can be optimized to **O(n)**.
5. Apply the sliding window technique to other problems involving arrays and strings.

## 7. Time Complexity:

$O(n)$ because each character is processed efficiently.

## 8. Space Complexity:

$O(k)$, where $k$ is the number of distinct characters stored in the map.